The Collections Framework

The Java Collections Framework provides a set of **interfaces** (like List, Set, and Map) and a set of **classes** (ArrayList, HashSet, HashMap, etc.) that implement those interfaces.

All of these are part of the java.util package.

They are used to store, search, sort, and organize data more easily - all using standardized methods and patterns.

Core Interfaces in the Collections Framework

Here are some common interfaces, along with their classes:

Interface	Common Classes	Description
`List`	`ArrayList`, `LinkedList`	Ordered collection that allows duplicates
`Set`	`HashSet`, `TreeSet`, `LinkedHashSet`	Collection of unique elements
`Map`	`HashMap`, `TreeMap`, `LinkedHashMap`	Stores key-value pairs with unique keys

Overview of Classes

The table below gives an overview of the common data structure classes and their characteristics:

Interface	Class	Description
`List`	`ArrayList`	Resizable array that maintains order and allows duplicates
	`LinkedList`	List with fast insert and remove operations
`Set`	`HashSet`	Unordered collection of unique elements
	`TreeSet`	Sorted set of unique elements (natural order)
	`LinkedHashSet`	Maintains the order in which elements were inserted
`Map`	`HashMap`	Stores key/value pairs with no specific order
	`TreeMap`	Sorted map based on the natural order of keys
	`LinkedHashMap`	Maintains the order in which keys were inserted

Java List Interface

The List interface is part of the Java Collections Framework and represents an **ordered collection** of elements.

You can access elements by their index, add duplicates, and maintain the insertion order.

Since List is an interface, you cannot create a List object directly.

Instead, you use a class that implements the List interface, such as:

- ArrayList - like a resizable array with fast random access
- LinkedList - like a train of cars you can easily attach or remove

Common List Methods

Method	Description
`add()`	Adds an element to the end of the list
`get()`	Returns the element at the specified position
`set()`	Replaces the element at the specified position
`remove()`	Removes the element at the specified position
`size()`	Returns the number of elements in the list

List vs. Array

Array	List
Fixed size	Dynamic size
Faster performance for raw data	More flexible and feature-rich
Not part of Collections Framework	Part of the Collections Framework

Java ArrayList

An ArrayList is like a resizable array. It is part of the java.util package and implements the List interface.

The difference between a built-in array and an ArrayList in Java, is that the size of an array cannot be modified (if you want to add or remove elements to/from an array, you have to create a new one). While elements can be added and removed from an ArrayList whenever you want.

import java.util.ArrayList; // Import the ArrayList class

ArrayList<String> cars = new ArrayList<String>(); // Create an ArrayList object

```
import java.util.ArrayList;

public class ArrayListClass {

        public static void main(String[] args) {
```

```
            ArrayList<String> cars = new ArrayList<String>();
        cars.add("Volvo");
        cars.add("BMW");
        cars.add("Ford");
        cars.add("Mazda");
        System.out.println(cars);  // [Volvo, BMW, Ford, Mazda]

        cars.add(0,"Huyndai");   // Add new car at first position
        System.out.println(cars); // [Huyndai, Volvo, BMW, Ford, Mazda]

        cars.get(0);  // Get the first element
        System.out.println(cars.get(0)); // Huyndai

        cars.set(0, "Opel"); //To modify an element, use the set() method and refer to the
index number:
        System.out.println(cars);  // [Opel, Volvo, BMW, Ford, Mazda]

        cars.remove(0);  //To remove an element, use the remove() method and refer to the
index number:
        System.out.println(cars);  // [Volvo, BMW, Ford, Mazda]

        cars.size(); //To find out how many elements an ArrayList have, use the size method:
        System.out.println(cars.size()); //4

        cars.clear(); // To remove all the elements in the ArrayList, use the clear() method:
        System.out.println(cars.size()); // 0
        System.out.println(cars); // [ ]


    }


}
```

- **Looping ArrayList**

```
import java.util.ArrayList;

public class LoopThroughanArrayList {

    public static void main(String[] args) {
            ArrayList<String> cars = new ArrayList<String>();
            cars.add("Volvo");
            cars.add("BMW");
            cars.add("Ford");
            cars.add("Mazda");
//          for (int i = 0; i < cars.size(); i++) {
//                  System.out.println(cars.get(i));
//
```

```
//                  }
                    // for each loop

                    for(String car:cars) {
                            System.out.println(car);
                    }

                    ArrayList<Integer> myNumbers = new ArrayList<Integer>();
            myNumbers.add(10);
            myNumbers.add(15);
            myNumbers.add(20);
            myNumbers.add(25);
            for (int i : myNumbers) {
              System.out.println(i);
            }
          }
}
```

- **Sorting ArrayList in Ascending and Descending order:**

```
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

public class SortArrayList {

        public static void main(String[] args) {
//      List<String> cars = new ArrayList<>();  //Sometimes you will see both List and
ArrayList in Java code
                    ArrayList<String> cars = new ArrayList<>();
                    cars.add("Suzuki");
                    cars.add("Volvo");
                    cars.add("BMW");
                    cars.add("Ford");
                    cars.add("Mazda");

                    Collections.sort(cars); //Ascending order
                    System.out.println(cars); // [BMW, Ford, Mazda, Suzuki, Volvo]


                    Collections.sort(cars, Collections.reverseOrder()); //Descending order
                    System.out.println(cars);  // [Volvo, Suzuki, Mazda, Ford, BMW]


                    }

}
```

Java LinkedList

The LinkedList class is almost identical to the ArrayList.

```
import java.util.LinkedList;

public class Main {

public static void main(String[] args) {

LinkedList<String> cars = new LinkedList<String>();

 cars.add("Volvo");

 cars.add("BMW");

cars.add("Ford");

cars.add("Mazda");

 System.out.println(cars);

}

}
```

ArrayList vs. LinkedList

The LinkedList class is a collection which can contain many objects of the same type, just like the ArrayList.

The LinkedList class has the same methods as ArrayList because both follow the List interface. This means you can add, change, remove, or clear elements in a LinkedList just like you would with an ArrayList.

However, while the ArrayList class and the LinkedList class can be used in the same way, they are built very differently.

How the ArrayList works

The ArrayList class has a regular array inside it. When an element is added, it is placed into the array. If the array is not big enough, a new, larger array is created to replace the old one and the old one is removed.

How the LinkedList works

The LinkedList stores its elements in "containers." The list has a link to the first container and each container has a link to the next container in the list. To add an element to the list, the

element is placed into a new container and that container is linked to one of the other containers in the list.

When To Use

Use an ArrayList for storing and accessing data, and LinkedList to manipulate data.

LinkedList Methods

For many cases, the ArrayList is more efficient as it is common to need access to random elements in the list, but the LinkedList provides several methods to do certain operations more efficiently:

Method	Description
`addFirst()`	Adds an element to the beginning of the list
`addLast()`	Add an element to the end of the list
`removeFirst()`	Remove an element from the beginning of the list
`removeLast()`	Remove an element from the end of the list
`getFirst()`	Get the element at the beginning of the list
`getLast()`	Get the element at the end of the list

```
import java.util.LinkedList;

public class LinkedListClass {
        public static void main (String [] args) {

        LinkedList<String> cars = new LinkedList<>();
        cars.add("Suzuki");
        cars.add("Volvo");
        cars.add("Huyndai");
        System.out.println(cars); //[Suzuki, Volvo, Huyndai]

        cars.addFirst("Mazda");  // Adds an element to the beginning of the list
        System.out.println(cars); //[Mazda, Suzuki, Volvo, Huyndai]

        cars.addLast("Jeep"); // Add an element to the end of the list
        System.out.println(cars);  // [Mazda, Suzuki, Volvo, Huyndai, Jeep]

        cars.getFirst();  // Get the element at the beginning of the list
        System.out.println(cars.getFirst()); // Mazda
```

```
        cars.getLast();  //        Get the element at the end of the list
        System.out.println(cars.getLast()); // Jeep


        cars.removeFirst();     //Remove an element from the beginning of the list
        System.out.println(cars); //[Suzuki, Volvo, Huyndai, Jeep]


        cars.removeLast();      //Remove an element from the end of the list
        System.out.println(cars); //[Suzuki, Volvo, Huyndai]
        }
}
```

Java Set Interface

The Set interface is part of the Java Collections Framework and is used to store a collection of **unique elements**.

Unlike a List, a Set does **not allow duplicates**, and it does not preserve the order of elements (unless you're using TreeSet or LinkedHashSet).

Common classes that implement Set:

- HashSet - fast and unordered
- TreeSet - sorted set
- LinkedHashSet - ordered by insertion

Tip: Use a Set when you need to store unique values only

Common Set Methods

Method	Description
`add()`	Adds an element if it's not already in the set
`remove()`	Removes the element from the set
`contains()`	Checks if the set contains the element
`size()`	Returns the number of elements
`clear()`	Removes all elements

Set vs. List

List	Set
Allows duplicates	Does not allow duplicates
Maintains order	Does not guarantee order
Access by index	No index-based access

A HashSet is a collection of elements where every element is **unique**.

```
HashSet<String> cars = new HashSet<>();
        cars.add("Honda");
        cars.add("Hyundai");
        cars.add("Volvo");
        cars.add("Honda"); // duplicate
        cars.add("Mazda");
        System.out.println(cars);

        System.out.println(cars.contains("Mazda"));   // To check whether an element exists in a HashSet

        System.out.println(cars.size()); //return size of set

        cars.remove("Volvo");
        System.out.println(cars); // Removes the element from the set


        cars.clear();  //To remove all elements, use the clear() method
        System.out.println(cars);
        System.out.println(cars.size());
```

Java TreeSet

A TreeSet is a collection that stores unique elements **in sorted order**.

It is part of the java.util package and implements the Set interface.

Tip: Unlike HashSet, which has no order, TreeSet keeps its elements sorted automatically.

```
    public static void main(String[] args) {
        TreeSet<String> cars = new TreeSet<>();
        cars.add("Volvo");
      cars.add("BMW");
```

```
    cars.add("Ford");
    cars.add("BMW");  // Duplicate
    cars.add("Mazda");

    System.out.println(cars);  //[BMW, Ford, Mazda, Volvo]

        cars.contains("Ford");   // Use contains() to check if an element exists:
        System.out.println(cars.contains("Ford"));  //true

        cars.remove("BMW");  // to remove element from set
        System.out.println(cars);  //[Ford, Mazda, Volvo]

}
```

Java LinkedHashSet

A LinkedHashSet is a collection that stores **unique elements** and **remembers the order they were added**.

It is part of the java.util package and implements the Set interface.

Tip: Use LinkedHashSet when you want a set that does not allow duplicates *and* keeps the original insertion order.

```
public static void main(String[] args) {

        LinkedHashSet<String> cars = new LinkedHashSet<>();
        cars.add("Honda");
        cars.add("Hyundai");
        cars.add("BMW");
        cars.add("Honda"); //duplicate

        System.out.println(cars);  //[Honda, Hyundai, BMW]

        System.out.println(cars.contains("BMW")); // to check existence

        cars.remove("BMW");
        System.out.println(cars); //[Honda, Hyundai]

}
```

Java Map Interface

The Map interface is a part of the Java Collections Framework and is used to store **key-value pairs**. Each key must be unique, but values can be duplicated.

A Map is useful when you want to associate a key (like a name or ID) with a value (like an age or description).

Common classes that implement Map:

- HashMap - fast and unordered
- TreeMap - sorted by key
- LinkedHashMap - ordered by insertion

Tip: Use a Map when you want to associate values with unique keys, like storing user IDs with names.

Common Map Methods

Method	Description
`put()`	Adds or updates a key-value pair
`get()`	Returns the value for a given key
`remove()`	Removes the key and its value
`containsKey()`	Checks if the map contains the key
`keySet()`	Returns a set of all keys

Map vs. Set vs. List

Feature	List	Set	Map
Duplicates allowed?	Yes	No	Keys: No Values: Yes
Stores key-value pairs?	No	No	Yes
Maintains order?	Yes	No (unless using TreeSet or LinkedHashSet)	No (unless using TreeMap or LinkedHashMap)

Java HashMap

A HashMap stores items in **key/value pairs**, where each key maps to a specific value.

It is part of the java.util package and implements the Map interface.

Instead of accessing elements by an index (like with ArrayList), you use a **key** to retrieve its associated **value**.

A HashMap can store many different combinations, such as:

- String keys and Integer values

- String keys and String values

```
HashMap<String, String> capitalCities = new HashMap<>();

        // Add keys and values (Country, City)
        capitalCities.put("England", "London");
        capitalCities.put("India", "New Dehli");
        capitalCities.put("Norway", "Oslo");
        capitalCities.put("Austria", "Wien");
        capitalCities.put("Norway", "Oslo"); // Duplicate
        capitalCities.put("USA", "Washington DC");

        System.out.println(capitalCities);// {Austria=Wien, USA=Washington
DC, Norway=Oslo, England=London, India=New
                                                                    // Dehli}

        System.out.println(capitalCities.keySet()); // [Austria, USA,
Norway, England, India] return only keys

        System.out.println(capitalCities.values()); // [Wien, Washington DC,
Oslo, London, New Dehli]

        System.out.println(capitalCities.containsKey("India")); // true

        System.out.println(capitalCities.containsValue("Oslo")); // true

        capitalCities.get("England"); // To access a value in the HashMap,
use the get() method and refer to its key:
        System.out.println(capitalCities.get("England")); // London

        capitalCities.remove("USA");
        System.out.println(capitalCities); // {Austria=Wien, Norway=Oslo,
England=London, India=New Dehli}

        capitalCities.clear();
        System.out.println(capitalCities); // to clear the map
```

Java TreeMap

A TreeMap is a collection that stores key/value pairs **in sorted order by key**.

It is part of the java.util package and implements the Map interface.

Tip: Unlike HashMap, which does not maintain order, TreeMap keeps its keys sorted.

```
public static void main(String[] args) {
        TreeMap<String, String> capitalCities = new TreeMap<>();
          capitalCities.put("England", "London");
          capitalCities.put("India", "New Dehli");
          capitalCities.put("Austria", "Wien");
          capitalCities.put("Norway", "Oslo");
          capitalCities.put("Norway", "Oslo"); // Duplicate
          capitalCities.put("USA", "Washington DC");
```

```
        System.out.println(capitalCities);  //{Austria=Wien, England=London, India=New Dehli, Norway=Oslo, USA=Washington DC}

        capitalCities.get("India");
        System.out.println(capitalCities.get("India")); //New Dehli

        System.out.println(capitalCities.remove("USA")); //Washington DC

        capitalCities.clear();
        System.out.println(capitalCities); // empty map
    }
```

Java LinkedHashMap

A LinkedHashMap stores keys and values, and keeps them in the same order you put them in.

It is part of the java.util package and implements the Map interface.

Tip: Use LinkedHashMap when you want predictable iteration order (insertion order).

```
LinkedHashMap<String, String> capitalCities = new LinkedHashMap<>();

        capitalCities.put("England", "London");
        capitalCities.put("India", "New Dehli");
        capitalCities.put("Austria", "Wien");
        capitalCities.put("Norway", "Oslo");
        capitalCities.put("Norway", "Oslo"); // Duplicate
        capitalCities.put("USA", "Washington DC");

        System.out.println(capitalCities);  // As per insertion order {England=London, India=New Dehli, Austria=Wien, Norway=Oslo, USA=Washington DC}

        System.out.println(capitalCities.get("India")); // New Dehli

        System.out.println(capitalCities.remove("USA")); //Washington DC

        capitalCities.clear();
        System.out.println(capitalCities); // Empty map
```

Java Iterator

An Iterator is an object that can be used to loop through collections, like ArrayList and HashSet.

It is called an "iterator" because "iterating" is the technical term for looping.

To use an Iterator, you must import it from the java.util package.

```
public static void main(String[] args) {
```

```
// Make a collection
    ArrayList<String> cars = new ArrayList<String>();
    cars.add("Volvo");
    cars.add("BMW");
    cars.add("Ford");
    cars.add("Mazda");

    // Get the iterator
    Iterator<String> it = cars.iterator();

    // Print the first item
    System.out.println(it.next());

 // Loop through a collection
    while(it.hasNext()) {
      System.out.println(it.next());
    }
}
```

ASHISH AMRUTE